

External Communication of CPU: With vs Without Cache

1. Without Cache (Direct Memory Access)

Communication Flow:

CPU $\longleftrightarrow$ Main Memory (RAM)

Steps:

1. CPU requests data (an instruction or operand).
2. Memory controller looks up the address in RAM.
3. Data is fetched and delivered to the CPU.

Problems Faced:

- **Slow Access Time:** RAM access latency is high compared to CPU speed.
 - **Memory Bottleneck:** Each request directly accesses RAM, increasing traffic.
 - **CPU Idle Time:** CPU remains idle waiting for slow memory fetches.
-

2. With Cache (Modern Architecture)

Communication Flow:

CPU $\longleftrightarrow$ Cache $\longleftrightarrow$ Main Memory

Cache Hierarchy:

- **L1 Cache:** Fastest, smallest, and built directly into CPU core.
- **L2 Cache:** Larger but slower than L1, often per core.
- **L3 Cache:** Shared across cores, slower but still faster than RAM.

Steps on a Data Request:

1. CPU checks **L1 cache**.
 - **Cache Hit:** Data found in cache $\Rightarrow$ access is immediate.
 - **Cache Miss:** Check L2 $\rightarrow$ L3 $\rightarrow$ RAM.
2. If fetched from RAM, data is stored in cache for future reuse.
3. This drastically reduces average memory access time.

Why Cache is Effective:

- Takes advantage of:
 - **Temporal Locality:** Recently accessed data is likely to be reused.
 - **Spatial Locality:** Nearby data is often accessed together.

3. Comparison Table: Cache vs No Cache

Feature	Without Cache	With Cache
Access Speed	Slow (RAM only)	Fast (L1 $\rightarrow$ L2 $\rightarrow$ RAM)
CPU Idle Time	High	Low
Power Consumption	High	Low (due to fewer RAM accesses)
System Cost	Lower	Higher (due to cache hardware)
Hit Rate	Not Applicable	High hit rate boosts performance

4. Analogy: Chef and Warehouse

Imagine the CPU is a chef, and RAM is a distant warehouse.

- **Without Cache:** The chef walks to the warehouse every time they need ingredients.
 - **With Cache:** The chef keeps a fridge (cache) beside them with commonly used items.
-

5. Unified Memory View with Cache

Concept:

Even though memory is split across cache and RAM, the CPU sees it as a unified memory address space.

$M(0)$ to $M(2^m - 1)$ — continuous from CPU's view

How It Works:

1. CPU requests data at address $M(x)$.
2. Cache controller checks:
 - **Hit:** Return data from cache in 1 cycle.
 - **Miss:** Fetch block from RAM and store in cache.
3. CPU accesses data normally — unaware of source.

This is called: *Seamless memory abstraction.*

6. Performance Implications

Source	Clock Cycles	Reason
L1 Cache	~ 1	Directly in CPU, very fast
L2/L3 Cache	5–20	Still fast, closer than RAM
Main Memory (RAM)	50–200+	Much slower, outside CPU

Conclusion: Cache significantly reduces memory latency, improving CPU throughput.

7. Example: Address $M(1000)$

Access Timeline:

- **If in Cache:** Fetched in 1 clock cycle.
- **If not in Cache:**
 - Fetch block from RAM (50–200 cycles).
 - Load into cache.
 - CPU accesses from cache afterward.

To the CPU, it looks like any other memory access.

8. Key Takeaways

- Caches hide memory latency and improve CPU performance.
- The memory hierarchy (L1, L2, L3, RAM) allows faster access to frequently used data.
- Efficient caching relies on:
 - **Temporal locality:** Reuse of recent data.
 - **Spatial locality:** Access of nearby memory locations.
- CPU treats cache and RAM as a single memory space.